Deep Oscillatory Neural Network : Supplementary Material

Nurani Rajagopal Rohan[1†], C Vigneswaran[1†], Sayan Ghosh[1, 3], Kishore Rajendran[2], A Gaurav[2], V. Srinivasa Chakravarthy[1, 4*]

[1*]Department of Biotechnology, Indian Institute of Technology Madras, Chennai, 600036, Tamil Nadu, India.
[2]Department of Electrical Engineering, Indian Institute of Technology Madras, Chennai, 600036, Tamil Nadu, India.
[4]Center for Complex Systems and Dynamics,Indian Institute of Technology Madras, Chennai, 600036, Tamil Nadu, India.
[3]Department of Computer Science and Engineering, SRM University-AP, Amaravati, 522502, Andhra Pradesh, India.

*Corresponding author(s). E-mail(s): schakra@ee.iitm.ac.in;
Contributing authors: nrrohan@gmail.com;
vigneswaran0610@gmail.com;
†These authors contributed equally to this work.

A Supplementary Results

A.1 Signal Generation

We begin with a simple demonstration of signal generation with DONN. The network is trained to produce a sinusoidal signal of a specific frequency when prompted by a specific label presented as input in 1-hot form. The labels are [1,2,3,4] and and the corresponding signal has frequency [1, 5, 7, 9] Hz. The size details of the network architecture are described in Table-A1. For this problem, the input is presented as $\mu(t)$. The network is able to generate the signals accurately (Fig. A1). The validation Mean Squared Error (MSE) loss is 0.01 ($p<0.05$, $n=10$).

Table A1: Network Architecture for Signal Generation task

Input frequency range	[1-9 Hz]
Initial frequency range of oscillators	[1-10 Hz]
Architecture	ReLU (10), Hopf (10), tanh (10) output (1)
Input type to oscillators	$\mu(t)$
Frequency of oscillators	Not trained

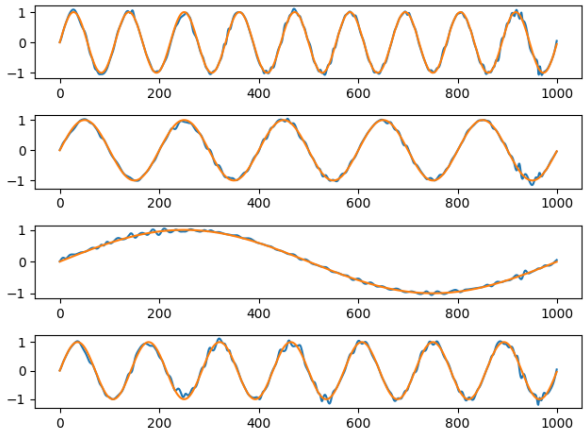


Fig. A1: The performance of the model for signal generation task. The model is able to generate the signals when presented with the required label.

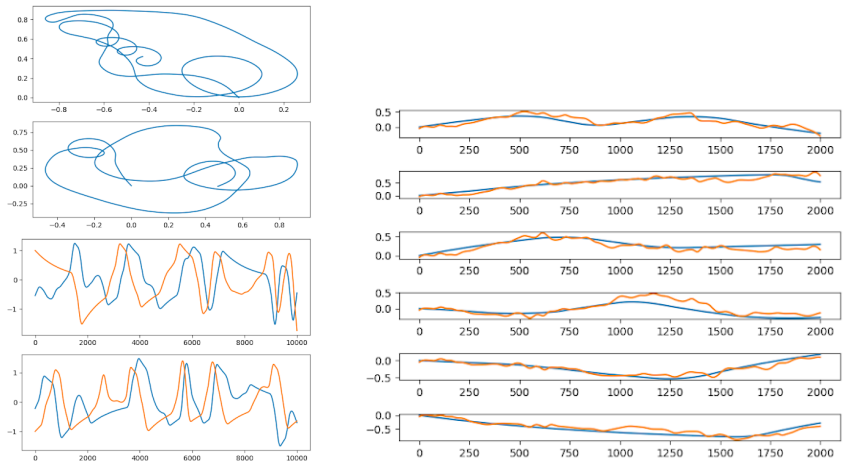
A.2 Learning Mathematical Operators

Data-2, We also apply the model on a data set of composed of trajectories where an agent is allowed to move in a bounded domain following an arbitrary velocity profile as described in [1]. The network is tasked to perform integration of the velocity profiles, $(\dot{x}, \dot{y})$ to produce the trajectory, (x, y), see Fig. A2a.

The size details of the network architecture for the this task is described in Table-A2. The network successfully performs the integration operation on the Data-2, see Fig. A2b. For integration, Data-2 validation MSE loss is 0.07 ($p<0.05$, $n=10$).

A.3 Video Frame Prediction Task

In the video frame prediction task, the network has to predict the next frame at time $t+1$, given the input frame at time t [2]. We generated a synthetic dataset to conduct the experiment. Each video in the dataset is of 40×40 greyscale images and the number



(a) (b)

Fig. A2: (a) Sample trajectory of the agent moving in a plane(above) and the corresponding velocity profiles (velocities along x and y axis) (below) for Data-2. The trajectories are generated using an algorithm defined in [1]. (b) The model is able to integrate the signals. (Blue) represents the desired and (orange) represents the model predicted output.

of frames is 16. Squares can move in any one of the possible directions in 360 degrees, where dimension of each square is in the range of 2 to 4, see Fig. A3. A total of 1000 such videos were generated and 8:2 train-validation split is used. The first 15 frames ($t = 0$ to $t = 14$) are given as the input to the network, and the network should predict the next 15 frames ($t = 1$ to $t = 15$). The network architecture is Table-A3. Adam optimiser with a learning rate of 0.001 is used to optimise the objective function and MSE was used as the loss function. The total trainable parameters are 30,401. The validation MSE loss after training is 0.04 ($p<0.05$,$n=10$). A sample prediction of our model is given in Fig. A3.

B Implementation details of DONN

- Data acquisition, pre-processing, and data loading. DONN was tested on standard benchmark datasets to compare and describe the baseline performance capabilities of our network. Some synthetically generated data sets were also used to facilitate the interpretation of the network. Consequently, no further data pre-processing was done. The details of all the datasets used can be found in our Data Availability Statement. The data were split to enable robust training, with separate validation and test sets providing an assessment of the network's generalization capabilities. The task relevant details are provided in the manuscript.
- Definition of the model. The architectures associated with each task are summarized in tabular form, reflecting a consistent motif of alternating static and oscillatory

Table A2: Network architecture for Learning Mathematical Operations

	Data 2
Input frequency range	Layer-1 : [0.1-10 Hz] Layer-2 : [0.1-2 Hz]
Initial frequency range of oscillators	[1-10 Hz]
Architecture	ReLU (20), Hopf (20), ReLU (20), Hopf (20), tanh (20), output (1)
Input type to oscillators	$I(t)$
Frequency of oscillators	Trained

Table A3: Video Frame Prediction

Initial frequency range of oscillators Input type to oscillator	[0.1-7 Hz] $I(t)$
Architecture	2 x OCNN (3x3,40 filters), flatten, output (2)
Frequency of oscillators	Trained

layers. The static layers (e.g., layer of ReLU, tanh, sigmoidal neurons) follow standard TensorFlow/PyTorch implementation. As described in the manuscript, there are three modes by which the input can be presented to our neural oscillatory layer − resonator mode, amplitude modulation and frequency modulation, see Fig. 10 of the main manuscript. Each implementation differs only in the dynamical equations that are numerically integrated to produce neural activation. For simplicity, we restricted our approach to the Euler numerical integration procedure for all three input modes. The model follows the conventional paradigm whereby activations for all units in a layer, and for all inputs in a batch, are evaluated simultaneously.

- Initialization. The weights were initialized using the backend's default activation-dependent scheme; specifically, we employed glorot_uniform. The initialization of the hyperparameter for the neural Hopf layers depend on the mode of input selected. Specifically, for the resonator mode of input; the oscillators are most sensitive and deterministic to the input when the input signal frequency is close to its natural frequency. And so, the oscillator frequencies are initialized to support this realization. For amplitude and frequency modulation modes, the initialization is meant to ensure standard modulation practices.
- Optimiser, Loss, metric evaluation. The default Adam($\beta_1 = 0.9, \beta_2 = 0.999$) optimizer is used across all task implementations. The loss is chosen to be Mean

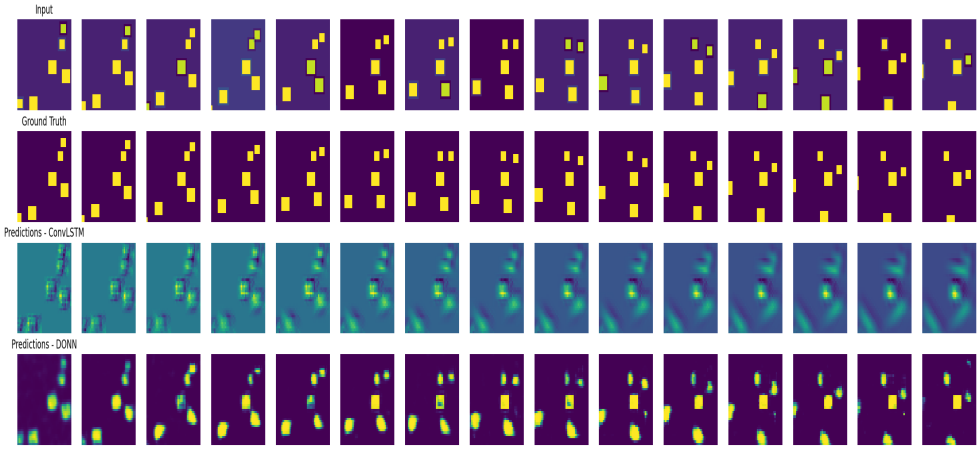


Fig. A3: Each row in the figure shows frames from t=0 to 15. Four rows are used to denote frames that are input to the model, desired output, output from an equivalent ConvLSTM layer, and the output from our network. Apart from lesser prediction loss value, the prediction by the Oscillator network visually look better.

Squared Error (MSE) across all tasks. The primary metric evaluated and reported are accuracy and loss, whichever suitable. The modified implementation of accuracy, necessitated by the time series nature of the system output, is described in Section 3.2 of the manuscript.

- Training was performed using standard backpropagation. The network was trained for a task-specific minimum number of epochs, with checkpoints saved after each epoch and validation performance used for early stopping. Multiple independent trials with different random seeds were conducted, and statistical estimates were reported.

C Hardware benchmarking by increasing number of layers and dataset size

We have benchmarked DONN with increasing the number of layers iteratively from 1 to 10 layers. A single layer consists of sequence of complex-valued weights, followed by hopf oscillators and complex ReLU activation. Source code for this analysis is available in the code repository. Input complex-float32 data of shape (10, 1000, 100) in the format (batch size, number of steps, and dimension) is given as input and the experiment is repeated for 10 times. The result of this analysis is shown in Fig. C1 below in terms of execution time and the number of trainable parameters. It can be inferred from the plots that the execution time increases linearly ($\mathcal{O}(n)$) with increasing layers, thus indicating stable execution-time characteristics.

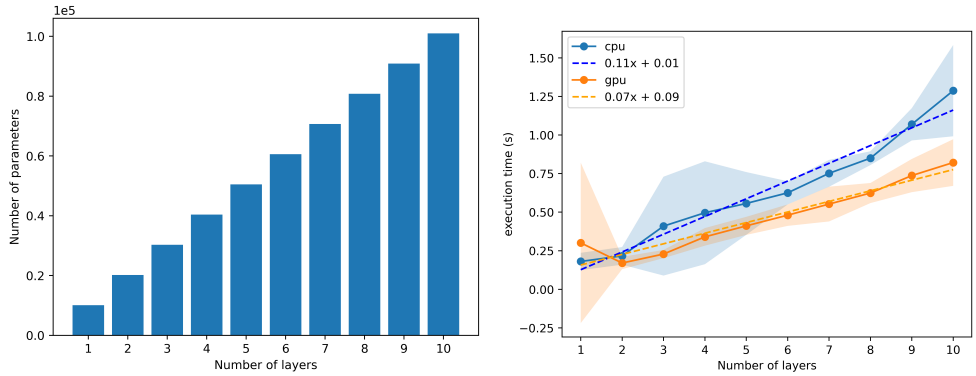


Fig. C1: Benchmarking performance of DONN by increasing the depth of the network in terms of execution time (in both CPU and GPU) and number of trainable parameters.

Similarly, to show that DONN exhibits stable execution-time characteristics with increasing dataset size, we performed a similar analysis. The input data size is progressively increased and the execution time in CPU is noted. Fig. C2 shows the result of the analysis, and it follows the desirable characteristic $\mathcal{O}(n)$.

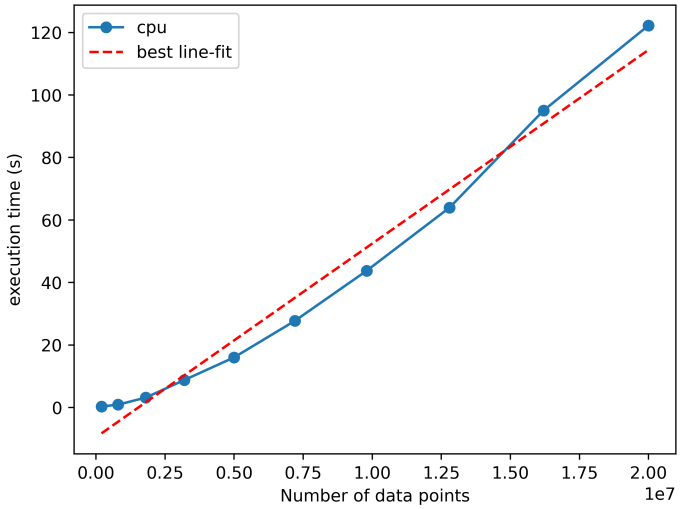


Fig. C2: Benchmarking performance of DONN by increasing data size in terms of execution time (CPU)

References

[1] Soman, K., Muralidharan, V. & Chakravarthy, V. S. A model of multisensory integration and its influence on hippocampal spatial cell responses. *IEEE Transactions on Cognitive and Developmental Systems* **10**, 637–646 (2018).

[2] Chang, Z. *et al.* Mau: A motion-aware unit for video prediction and beyond. *Advances in Neural Information Processing Systems* **34**, 26950–26962 (2021).